

LINQ ToList and ToArray Methods in C# with Examples

In this article, I will discuss the **LINQ ToList and ToArray Methods in C#** with Examples. Please read our previous article discussing the [LINQ Zip Method in C#](#) with Examples. The LINQ ToList and ToArray Methods in C# belong to the conversion operator category.

The LINQ ToList and ToArray methods in C# are used to convert collections or sequences to a List<T> or an array T[], respectively. Both methods are part of the System.Linq namespace and can be applied to any type implementing IEnumerable<T>. These methods are commonly used for materializing query results into concrete collections.

LINQ ToList Method in C#:

The LINQ ToList Method in C# creates a List<T> from an IEnumerable<T>. This is particularly useful when you need a list with the functionality it provides, like the ability to add or remove items. This method creates a System.Collections.Generic.List<T> collection from a System.Collections.Generic.IEnumerable<T>. This method causes the query to be executed immediately. The signature of the ToList method is shown below.

```
public static List<TSource> ToList<TSource>(this IEnumerable<TSource> source);
```

Parameters:

- **source:** The data type of source is [System.Collections.Generic.IEnumerable<T>](#).

Type parameters:

- **TSource:** The type of elements contained in the source sequence.

Returns:

- It returns [System.Collections.Generic.List<T>](#), which contains elements from the source sequence.

Exceptions:

- This method throws [System.ArgumentNullException](#) when the source sequence is null.

Example to Convert int array to List<int> using the LINQ ToList Method in C#

In the following example, we first create an integer array and then convert that integer array into a list (i.e., List<int>) by using the LINQ ToList Method in C#.

```
using System.Collections.Generic;  
using System.Linq;  
using System;
```

```

namespace LINQToListMethodDemo
{
    class Program
    {
        public static void Main()
        {
            //Creating Integer Array
            int[] numbersArray = { 10, 22, 30, 40, 50, 60 };
            //Converting Integer Array to List using ToList method
            List<int> numbersList = numbersArray.ToList();
            //Accessing the List Elements
            foreach (var num in numbersList)
            {
                Console.Write($"{num} ");
            }
            Console.ReadKey();
        }
    }
}

```

Output: 10 22 30 40 50 60

Working with Complex Type:

Let us see how we can work with the ToArray method of a collection of Complex Types. For this, we are going to use the following Employee class. So, create a class file named **Employee.cs** and copy and paste the following code. It's a very simple class, having only three properties.

```

namespace LINQToArrayMethodDemo
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public string Department { get; set; }
    }
}

```

Next, modify the Main method of the Program class as follows. As you can see, first, we are creating an array of Employees and storing 5 employee information. Then, we convert that array of Employees to a list of Employees by calling the ToList Method.

```

using System.Collections.Generic;
using System.Linq;
using System;
namespace LINQToArrayMethodDemo
{
    class Program
    {
        public static void Main()

```

```

{
    //Create an Array of Employees
    Employee[] EmployeesArray = new Employee[]
    {
        new Employee() {ID = 1, Name = "Pranaya", Department = "IT" },
        new Employee() {ID = 2, Name = "Priyanka", Department = "HR" },
        new Employee() {ID = 3, Name = "Preety", Department = "HR" },
        new Employee() {ID = 4, Name = "Sambit", Department = "IT" },
        new Employee() {ID = 5, Name = "Sudhanshu", Department = "IT"}
    };
    //Converting Array to List
    List<Employee> EmployeesList = EmployeesArray.ToList();
    //Accessing the Elements of the List
    foreach (var emp in EmployeesArray)
    {
        Console.WriteLine($"ID: {emp.ID}, Name: {emp.Name}, Department
{emp.Department}");
    }
    Console.ReadKey();
}
}
}

```

Output:

```

ID: 1, Name: Pranaya, Department: IT
ID: 2, Name: Priyanka, Department: HR
ID: 3, Name: Preety, Department: HR
ID: 4, Name: Sambit, Department: IT
ID: 5, Name: Sudhanshu, Department: IT

```

What happens when the Source Array is Null?

When the Source Array is Null, and if we try to convert that null array into a list using the ToList method, we will get one runtime exception saying the value cannot be null. For a better understanding, please have a look at the following example. In the example below, we create one null array and then convert that integer null array to a list.

```

using System.Collections.Generic;
using System.Linq;
using System;
namespace LINQToListMethodDemo
{
    class Program
    {
        public static void Main()
        {
            //Creating Integer Array and Initializing it with NULL

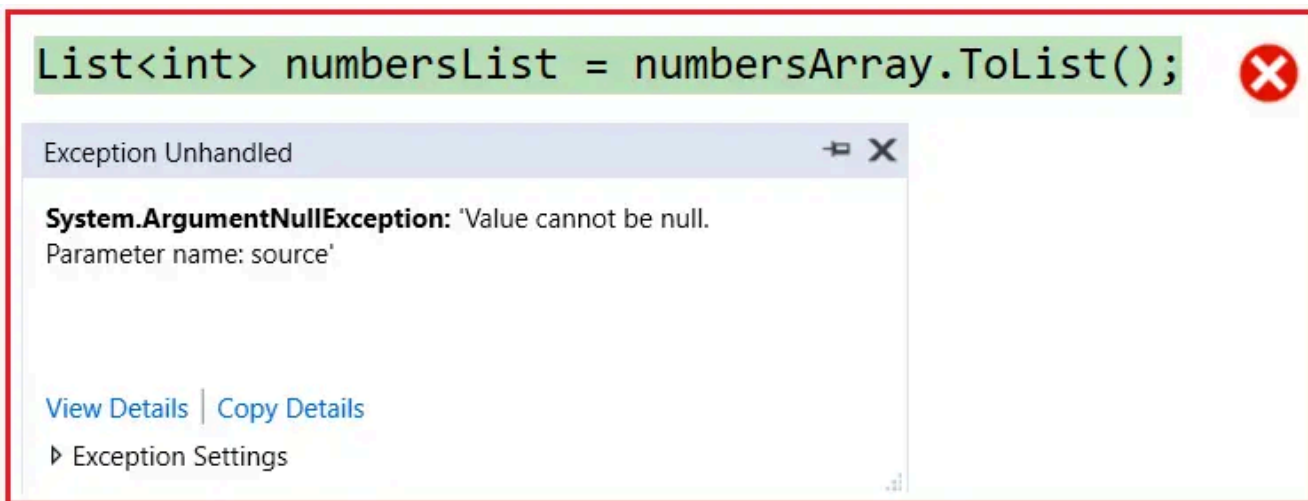
```

```

    int[] numbersArray = null;
    //Converting Integer Array to List using ToList method
    List<int> numbersList = numbersArray.ToList();
    //Accessing the List Elements
    foreach (var num in numbersList)
    {
        Console.WriteLine($"{num} ");
    }
    Console.ReadKey();
}
}
}

```

You will get the following exception at runtime when you run the above code.



LINQ ToArray Method in C#:

The LINQ ToArray Method converts an `IEnumerable<T>` to an array `T[]`. This is useful when you need a fixed-size collection or interfacing with APIs requiring arrays. This method copy the elements of the `System.Collections.Generic.List<T>` to a new array. This method causes the query to be executed immediately. The signature of this method is shown below.

```
public T[] ToArray();
```

Here, T is the array type, and this method converts a list into an array and returns that array containing copies of the elements of the `System.Collections.Generic.List<T>`.

Example: Convert List<int> to Integer Array in C# using LINQ ToArray Method.

In the following example, first, we create a list of integers and then convert that list of integers into an integer array (i.e., `int[]`) by using the LINQ ToArray Method in C#. The following code is self-explained, so please go through the comment lines for a better understanding.

```

using System.Collections.Generic;
using System;
namespace LINQToArrayMethodDemo
{
    class Program
    {
        public static void Main()
        {
            //Create a List
            List<int> numbersList = new List<int>()
            {
                10, 22, 30, 40, 50, 60
            };
            //Converting List to Array
            int[] numbersArray = numbersList.ToArray();
            //Accessing the Elements of the Array
            foreach (var num in numbersArray)
            {
                Console.Write($"{num} ");
            }
            Console.ReadKey();
        }
    }
}

```

Output: 10 22 30 40 50 60

Working with Complex Type:

We are going to work with the same Employee class. So, modify the Main method of the Program class as follows. As you can see, first, we are creating a list collection of type Employees and storing 5 employee information. Then, we convert that list of Employees to an array of Employees by calling the ToArray Method.

```

using System.Collections.Generic;
using System;
namespace LINQToArrayMethodDemo
{
    class Program
    {
        public static void Main()
        {
            //Create a List of Employees
            List<Employee> EmployeesList = new List<Employee>()
            {
                new Employee() {ID = 1, Name = "Pranaya", Department = "IT" },
                new Employee() {ID = 2, Name = "Priyanka", Department = "HR" },
                new Employee() {ID = 3, Name = "Preety", Department = "HR" },
                new Employee() {ID = 4, Name = "Sambit", Department = "IT" },
                new Employee() {ID = 5, Name = "Sudhanshu", Department = "IT"}
            };
        }
    }
}

```

```

//Converting List to Array
Employee[] EmployeesArray = EmployeesList.ToArray();
//Accessing the Elements of the Array
foreach (var emp in EmployeesArray)
{
    Console.WriteLine($"ID: {emp.ID}, Name: {emp.Name}, Department
{emp.Department}");
}
Console.ReadKey();
}
}
}

```

Output:

```

ID: 1, Name: Pranaya, Department: IT
ID: 2, Name: Priyanka, Department: HR
ID: 3, Name: Preety, Department: HR
ID: 4, Name: Sambit, Department: IT
ID: 5, Name: Sudhanshu, Department: IT

```

What Happens When We Call the ToArray Method on a Null List?

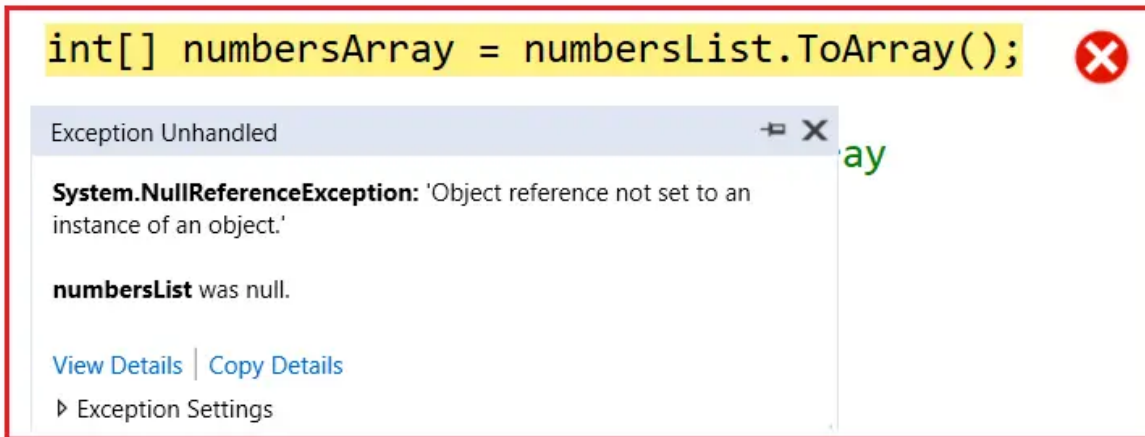
We will get a Null Reference Exception if we call the ToArray Method on a Null List. For a better understanding, please have a look at the following example. In the below example, we are creating one list, initializing that list with NULL, and then calling the ToArray method on that list instance.

```

using System.Collections.Generic;
using System;
namespace LINQToArrayMethodDemo
{
    class Program
    {
        public static void Main()
        {
            //Create a List
            List<int> numbersList = null;
            //Converting List to Array
            int[] numbersArray = numbersList.ToArray();
            //Accessing the Elements of the Array
            foreach (var num in numbersArray)
            {
                Console.Write($"{num} ");
            }
            Console.ReadKey();
        }
    }
}

```

When you run the above code, you will get the following Exception.



Key Points

- **Materialization:** ToList and ToArray are used for materializing the results of a LINQ query. This means they execute the query and create a concrete collection with the results.
- **Immutability of Results:** The resulting List<T> or T[] is a snapshot of the data during the call. Subsequent changes to the source collection do not affect the list's contents or array.
- **Memory Allocation:** Both methods allocate memory for the new collection. This can be a consideration for very large datasets.
- **Choice of Method:** Use ToList when you need the dynamic features of a list (like adding or removing items), and use ToArray when you need a fixed-size, immutable collection or are interfacing with APIs that require arrays.

Both ToList and ToArray are commonly used in scenarios where you need to work with the results of a LINQ query as a standard collection, like in data-binding scenarios or when passing data to methods that require specific collection types.

In the next article, I will discuss the [LINQ ToDictionary Method in C#](#) with Examples. In this article, I explain the need and use of **LINQ ToList and ToArray Methods in C#** with Examples. I hope you enjoy this LINQ ToList and ToArray Methods in C# article.